

面试手册

Redis

把基础，读成底气。

1 个科目 / 20 道问题

回答要点 • 常见追问 • 易错点 • 参考来源

整理日期：2026-10-09

本手册总结经典面试知识点，不是全网公司面经的穷举，也不代表真实招聘频次统计。题解为独立中文整理，具体版本与实现请参考题目中的原始资料。

C++ 以 C++17/20 为主；MySQL 以 8.4 LTS InnoDB 为主；Redis 以开源版常用功能为主。目录、PDF 书签及参考链接均可点击。

阅读目录

阅读目录	2
Redis	3
01. Redis 常用数据类型分别适合什么场景?	3
02. Redis 的类型与底层编码有什么区别?	3
03. Redis 是单线程吗, 为什么能很快?	4
04. 单条命令原子是否意味着业务操作原子?	5
05. MULTI、EXEC、WATCH 能提供什么保证?	6
06. Lua 脚本的原子性与风险是什么?	6
07. RDB 快照如何工作, 有什么代价?	7
08. AOF、重写与混合持久化有什么关系?	8
09. 过期 key 如何删除, TTL 有哪些细节?	9
10. 内存淘汰策略如何选择?	9
11. 主从复制、全量与增量同步如何工作?	10
12. Sentinel 解决什么问题, quorum 是什么?	11
13. Cluster 的 16384 个槽与 hash tag 怎么用?	11
14. 缓存穿透、击穿与雪崩如何区分?	12
15. MySQL 与 Redis 缓存如何保持一致?	13
16. Redis 分布式锁怎么写, 有哪些不能保证的事?	13
17. Pipeline 为什么提速, 和事务有何区别?	14
18. Stream 消费者组如何实现确认与重试?	15
19. 大 key 与热 key 如何发现和治理?	15
20. 如何评估 Redis 的可靠性与尾延迟?	16

Redis

缓存、持久化与分布式边界

以 Redis Open Source 为范围，覆盖常用结构、原子操作、复制与 Cluster，以及缓存一致性和锁的真实保证边界；涉及新能力时注明版本差异。

01. Redis 常用数据类型分别适合什么场景？

数据类型与执行 / 基础

String 适合值缓存与计数，Hash 适合字段对象，List 适合顺序列表，Set 适合去重集合，Sorted Set 适合带分数排名。选型应围绕访问操作、规模和可靠性要求，不只记住五个名字。

回答要点

1. String 保存字节序列，可存文本、序列化对象或整数计数；Hash 把对象拆成字段便于局部读取和修改。大对象整体反序列化与多个小字段访问各有成本，应根据读取和更新粒度选择。
2. List 支持头尾操作，Set 支持成员判断及集合运算，Sorted Set 按分数排序且成员唯一。排行榜需要考虑同分排序、分页成本和分数更新，集合交并运算要注意参与集合规模。
3. Bitmap、HyperLogLog、GEO 与 Stream 等扩展了场景；版本和部署还影响 JSON、概率结构等能力。HyperLogLog 给出近似去重计数，Stream 支持消费状态，不能把所有结构都当精确集合使用。

常见追问

追问：排行榜为什么不用 List？List 保存位置，但插入或更新某用户排名需要寻找和移动逻辑；Sorted Set 可以按成员更新分数并按分数取区间。最终复杂度还取决于命令和返回数量，不能笼统说所有 ZSet 操作都是常数时间。

易错点

类型是对外语义，底层编码是实现，两者不可混为一谈。用 List 做队列还要另行考虑取出后崩溃、确认、重试与消息保留。

延伸阅读 / 原始参考资料

[Redis 官方](#) • 数据类型

[Redis 官方](#) • 有序集合

02. Redis 的类型与底层编码有什么区别？

数据类型与执行 / 进阶

同一数据类型可按内容和规模采用不同编码。紧凑编码减少对象与指针开销，达到阈值后会转换；讨论 `listpack`、`intset` 或跳表时应注明版本，不能沿用旧 `ziplist` 结论描述所有实例。

回答要点

1. 小 Hash 和小 Sorted Set 在新版本常使用 `listpack` 等紧凑结构，节省大量独立分配与指针空间。阈值受元素数及值大小配置影响，具体编码可通过 `OBJECT_ENCODING` 观察而非猜测。
2. 整数集合、哈希表或有序集合的字典与跳表服务不同访问需求。紧凑结构有时需要扫描或移动数据，规模扩大后会转换，优化的本质是在小规模内存收益和操作成本之间平衡。
3. Redis 7 前后的压缩列表演进、Redis 7.2 的集合紧凑编码变化都说明实现有版本边界。不要为省内存无限调大紧凑阈值，编码转换和线性操作可能扩大尾延迟，应做代表性负载测试。

常见追问

追问：为什么一万个小 key 比一个 Hash 更占内存？独立 key 需要重复字典条目、对象及分配元数据，聚合可摊薄部分开销。不过也会让过期、分片和大对象操作粒度变粗，不能只看省下的字节就无条件合并。

易错点

不要把 `ziplist` 当当前所有集合的统一底层实现。也不要认为一旦采用紧凑编码就必然更快，它主要优化小对象空间，访问复杂度仍需看具体命令。

延伸阅读 / 原始参考资料

Redis 官方 • 内存优化

Redis 官方 • 编码观察

03. Redis 是单线程吗，为什么能很快？

数据类型与执行 / 基础

Redis 核心命令通常在主执行线程按序处理，但进程还包含后台任务和可配置的 I/O 线程。内存访问、高效结构和事件驱动降低开销；慢命令仍会阻塞其他请求，因此“全部只有一个线程”不准确。

回答要点

1. 大量工作是短小的内存结构操作，事件循环复用网络连接，减少频繁线程切换和锁竞争。性能还依赖命令复杂度、值大小、网络往返与硬件，不能脱离负载承诺固定每秒请求数。
2. 后台线程可承担 AOF 同步、异步释放等工作，部分版本支持网络 I/O 线程；持久化重写或快照还可能使用子进程。I/O 并行并不意味着所有修改数据的命令都在多个线程并行执行。
3. 单个长脚本、巨大集合运算或大返回值会占用主执行路径，让其他请求排队。扩展可通过减少单次工作、批量网络传输以及多实例分片实现，单纯增加客户端并发并不会消除热点。

常见追问

追问：开启 I/O 线程能解决慢 Lua 吗？不能。I/O 线程主要改善网络读写相关成本，耗时脚本仍占用数据执行路径。应缩短脚本、限制处理数量或改造数据模型，同时用延迟和命令统计判断瓶颈是否真在网络。

易错点

不能用“单线程所以不可能并发问题”推导应用安全。多个请求仍会交错，客户端多命令流程、持久化和故障切换也都有独立一致性边界。

延伸阅读 / 原始参考资料

Redis 官方 • 延迟诊断

Redis 官方 • 线程与运行信息

04. 单条命令原子是否意味着业务操作原子？

数据类型与执行 / 基础

INCR 等命令的读改写在 Redis 内部完成，其他请求不会插在该命令中间；客户端分开 GET、计算、SET 则会发生交错。原子执行只描述可观察的命令边界，不自动包含持久化或跨系统业务成功。

回答要点

1. 两个客户端先读取同一个计数，再各自写入加一结果会丢失一次更新。用 INCR 可把读取和更新合成服务端操作，但它要求值可按整数解释，数值范围和错误返回仍要正确处理。
2. INCR 后再单独 EXPIRE 存在客户端崩溃后未设置 TTL 的窗口，且反复刷新 TTL 可能改变限流语义。应在短 Lua、事务或适合的命令组合中统一定义计数及过期规则。
3. 超时只说明客户端没拿到确定结果，命令可能已经执行。盲目重试计数、扣减会重复生效；若要求业务幂等，需要请求标识与去重状态，或让权威数据库约束最终结果。

```
INCR page:views
SET cache:user:42 '{"name":"Lin"}' EX 300
```

常见追问

追问：Redis 原子扣库存就能保证不超卖吗？它可在单实例内原子检查并扣减，但预热库存、重复请求、故障丢写和数据库落账仍可能破坏业务约束。需定义幂等、补偿与最终库存的权威来源，不能仅靠一条命令完成全链路保证。

易错点

原子性、隔离性和持久性不是一个概念。命令执行期间没有交错，不等于断电不会丢失，也不等于多个 Redis 节点或数据库修改同时提交。

延伸阅读 / 原始参考资料

Redis 官方 • 原子计数

05. MULTI、EXEC、WATCH 能提供什么保证？

数据类型与执行 / 进阶

MULTI 把命令排队，EXEC 按序连续执行；WATCH 检查关注的键是否在提交前发生变化，提供乐观并发控制。Redis 事务不会像关系数据库那样自动回滚已成功的命令，必须检查每个返回结果。

回答要点

1. 排队期间命令返回 QUEUED，通常尚未执行；EXEC 执行时其他客户端命令不会插入该组执行。语法或排队错误可导致整组拒绝，运行时的类型错误则可能仅影响对应命令，其余继续执行。
 2. WATCH 在读取前监视键，再由客户端计算并在 MULTI/EXEC 中提交。监视状态变化会使 EXEC 失败，需要重新读取和有限重试；连接专属状态意味着使用连接池时要保证操作落在同一连接。
 3. 事务内无法把尚未执行的查询结果取回后继续客户端分支判断，因此复杂读改写常更适合短 Lua。
- Cluster 中相关多 key 通常需同槽，事务的连续执行保证也不提供跨槽两阶段提交。

```
WATCH balance
GET balance
MULTI
SET balance 90
EXEC
-- EXEC 若因监视键变化而失败，重新读取并计算
```

常见追问

追问：WATCH 失败说明出错了么？通常说明乐观条件已被其他修改打破，是预期冲突。应重新读取状态再计算，而非只重发旧写入。高争用场景可能频繁失败，需要退避、原子服务端操作或调整热点设计。

易错点

不要说 MULTI 是“执行失败全部回滚”。运行时错误不会撤销之前已成功的命令；DISCARD 是放弃尚未执行的队列，也不是回滚已执行写入。

延伸阅读 / 原始参考资料

Redis 官方 • 事务

Redis 官方 • WATCH

06. Lua 脚本的原子性与风险是什么？

数据类型与执行 / 进阶

Lua 能把校验与修改放到服务端连续执行，减少网络交错与往返。但脚本执行期间会占用执行路径，运行错误也不自动回滚此前写入。应限制工作量，传入显式键并处理脚本缓存失效。

回答要点

1. 适合比较锁令牌后删除、库存条件扣减或限制窗口计数等短逻辑。每个脚本都应先检查参数与数据类型，再执行必要写入，避免后续错误留下部分修改，且不要用无界循环处理整个数据库。
2. 键名通过 KEYS 显式传入，其他值通过 ARGV 传入，不要在脚本中拼出未声明访问键。这样客户端和集群可理解涉及的键；Cluster 多键脚本通常要求这些键属于同一 hash slot。
3. EVALSHA 只发送脚本摘要可减少重复传输，但脚本缓存不是永久资产，重启、切换或缓存清理后可能出现 NOSCRIPT。客户端应准备重新加载或 EVAL 回退，并留意特定版本的脚本缓存行为。

```
-- KEYS[1] 为锁, ARGV[1] 为本次持有者令牌
if redis.call('GET', KEYS[1]) == ARGV[1] then
    return redis.call('DEL', KEYS[1])
end
return 0
```

常见追问

追问：脚本超时会自动停止并回滚吗？不会把已发生的写入自动撤销。长脚本会造成忙状态，能否用 SCRIPT KILL 安全结束取决于是否已经写入；更可靠的做法是事先限制规模，并避免把耗时计算塞进 Redis。

易错点

“原子”表示执行时其他请求不交错，不代表事务式失败回滚。脚本内已经执行的写入，在后续命令报错时仍可能保留。

延伸阅读 / 原始参考资料

Redis 官方 • Lua 脚本

07. RDB 快照如何工作，有什么代价？

持久化与内存 / 基础

RDB 保存某一时刻的数据快照，文件紧凑，适合备份与较快装载。后台快照通常通过 fork 子进程生成，但 fork 暂停、写时复制及磁盘 I/O 仍会影响服务；故障会丢失快照之后的变化。

回答要点

1. BGSAVE 通常创建子进程读取快照视图，主进程继续处理请求；父子共享页面直到写时复制发生。数据集大且写入多时，额外内存和 fork 时间可能明显，后台不等于完全零阻塞。
2. 快照适合定期备份、复制传输和可容忍一定丢失窗口的场景。保存周期越长，未持久化修改越多；保存频繁则增加 fork 与 I/O 成本，应按恢复点目标和负载选择而非机械套配置。
3. 备份文件应复制到独立故障域并验证可装载，还要监控最近快照状态、耗时与磁盘空间。只有本机一份 RDB 无法应对整机或存储损坏，启用持久化也不等于已经具备恢复流程。

常见追问

追问：SAVE 和 BGSAVE 有什么区别？SAVE 在当前执行路径同步生成快照，过程中阻塞正常请求；BGSAVE 通常使用子进程，减少长时间阻塞但仍有 fork 等代价。生产选择还要看内存余量、写入强度及是否存在其他持久化任务。

易错点

不要承诺“RDB 完全不影响延迟”。大数据集 fork 与写时复制会带来暂停和内存压力；快照恢复能恢复某一时刻，不能重现其后未保存的写入。

延伸阅读 / 原始参考资料

Redis 官方 • 后台快照

Redis 官方 • 持久化概览

08. AOF、重写与混合持久化有什么关系？

持久化与内存 / 进阶

AOF 记录用于重建数据的写入，刷盘策略决定性能与丢失窗口；重写压缩冗余历史并生成等价恢复内容。Redis 7 起使用多部件 AOF，基础文件可采用 RDB 前缀，恢复时再应用增量日志。

回答要点

1. appendfsync always 强化提交落盘但增加同步成本，everysec 常在性能与丢失窗口间折中，no 依赖操作系统同步。everysec 通常讨论约一秒未刷盘风险，不应把它当任何故障下绝对最多一秒的契约。
2. 重写根据当前状态生成较精简的恢复表示，不是机械复制原 AOF。多部件 AOF 使用基础文件、增量文件与 manifest 管理切换，后台写入期间仍需维护新的变化，关注内存、磁盘与失败重试。
3. 同时启用 RDB 与 AOF 时，恢复策略通常优先使用更完整的 AOF。混合持久化把基础快照的装载优势与后续增量结合，但仍需正确管理相关文件，备份不能漏掉 manifest 所引用的组成部分。

常见追问

追问：AOF 重写是不是越频繁越好？不是。它减少恢复文件体积和重放工作，但也带来 fork、额外磁盘及写时复制压力。应根据日志增长比例、重写耗时、恢复目标和内存余量设置阈值，并监控失败与持续增长。

易错点

不要把 Redis 7 的 AOF 仍讲成永远只有一个文件，也不要将写入页缓存当成已经 fsync。日志追加、系统同步及备份保留是不同环节。

延伸阅读 / 原始参考资料

Redis 官方 • AOF 重写

Redis 官方 • 持久同步等待

09. 过期 key 如何删除，TTL 有哪些细节？

持久化与内存 / 基础

Redis 结合访问时的被动过期和周期性的主动抽样清理。TTL 到点表示键在语义上失效，不保证对应内存立即按毫秒释放；修改键是否保留过期时间取决于具体命令及选项。

回答要点

1. 访问已超时键时触发被动过期；主动清理会抽样检查带过期时间的键，以处理再也不被访问的键。主动任务要在回收和 CPU 之间平衡，大量键同刻到期可能形成清理与缓存回源压力。
2. SET 替换值通常会移除原 TTL，可使用 KEEPTTL 或显式 EX/PX 定义行为；HSET 等修改内容的命令一般会自动移除整个键的 TTL。应看命令语义，不能笼统说写入都会续期。
3. TTL 返回剩余秒数，PTTL 返回毫秒；无过期时间与键不存在使用不同的特殊返回值。Redis 以绝对时间处理过期信息，系统时钟异常、恢复与复制场景也需要结合实际配置理解。

常见追问

追问：能用过期通知做可靠定时任务吗？不应直接依赖。实际清理时间可能晚于 TTL，通知基于 Pub/Sub 且订阅断开会丢消息。可靠调度需要持久任务状态、轮询或队列、幂等执行与补偿机制，通知只能作为加速提示。

易错点

过期与内存淘汰不同：前者依据时间，后者依据容量和策略。也不要将键已不可读等同于其所有内存已经立即释放。

延伸阅读 / 原始参考资料

Redis 官方 • EXPIRE 与过期机制

Redis 官方 • 键空间通知

10. 内存淘汰策略如何选择？

持久化与内存 / 基础

达到 maxmemory 后 Redis 根据策略选择淘汰或拒绝部分写入。allkeys 面向全部键，volatile 仅面向带过期的键；LRU 关注近期使用，LFU 关注访问频率，具体实现采用近似方法而非完美排序。

回答要点

1. allkeys-lru 或 allkeys-lfu 常用于可重新构建的缓存；volatile 策略限定候选为设置过期的键，无合适候选时可能无法释放空间。不要把重要持久数据与可淘汰缓存混放后期待策略自动分辨。
2. noeviction 不主动淘汰，但达到限制后可能拒绝需要增加内存的写操作，读取仍可工作。调用方必须处理错误，否则缓存异常可能传播为业务失败；无限增大 maxmemory 会压缩系统与持久化余量。

3. maxmemory 的计算与系统实际内存占用不是完全同一个数字，复制及 AOF 缓冲等有特殊处理，fork 写时复制还需要额外空间。选择容量要给操作系统、碎片和后台任务保留余地，并监控淘汰率。

常见追问

追问：LRU 和 LFU 谁更好？近期热度变化快时 LRU 可能更适合，稳定的长期热点可能受益于 LFU。两者需用真实访问分布、命中率与尾延迟验证，不能只根据名称判断，LFU 也有计数近似和衰减行为。

易错点

不能认为设置 TTL 后键就只能因过期删除，淘汰策略可能提前删除它。反过来，持久化开启也不会阻止键按策略被淘汰并写入持久状态。

延伸阅读 / 原始参考资料

Redis 官方 • 内存淘汰

11. 主从复制、全量与增量同步如何工作？

复制与集群 / 进阶

Redis 默认异步复制，副本按主节点复制流更新。首次或无法补齐缺口时需全量同步，短暂断线在复制标识和 backlog 范围允许时可部分重同步。副本能扩读，但读取可能落后。

回答要点

1. 全量同步把当前数据基线传给副本，并在相应流程中追加期间变化；基线装载会消耗网络、磁盘或内存资源。无盘复制等配置改变传输路径，但不消除恢复数据集与传送增量的成本。
2. 部分同步依据复制标识和偏移，请求缺失的历史复制流。backlog 是有限缓冲，断线过久或写入速率太高可能超出保留范围，就需要全量同步，所以应结合峰值写入和允许断线时间设置容量。
3. 主库向客户端确认时，副本未必已经收到并应用写入。故障提升落后副本可能丢失已确认的修改；读副本可出现旧值，要求写后读一致的操作应选择主节点或明确等待与回退策略。

常见追问

追问：WAIT 可以保证强一致吗？它等待当前连接之前的写入达到指定副本确认数量，必须检查实际返回数量。它提高特定故障下的保留概率，但不是共识协议，也不保证所有切换情形都不丢写，不能将 Redis 变成强一致数据库。

易错点

不要把 replication backlog 与客户端输出缓冲混为一物。复制标识、偏移和保留窗口共同决定能否部分同步，连接重建成功不等于数据已经追平。

延伸阅读 / 原始参考资料

Redis 官方 • 复制

Redis 官方 • 副本确认等待

12. Sentinel 解决什么问题，quorum 是什么？

复制与集群 / 进阶

Sentinel 监控非分片主从部署、判断故障、协调提升副本并通知客户端。quorum 用于客观故障判断，真正执行故障切换还需要多数 Sentinel 授权；它提供高可用管理，不消除异步复制的丢写窗口。

回答要点

1. 单个 Sentinel 的超时判断属于主观下线，多个监控者达到配置 quorum 才能形成客观下线。故障切换还涉及领导者选择和多数授权，因此把 quorum 简单设置为一并不能绕过多数机制。
2. 通常至少部署三个 Sentinel 并分散故障域，客户端通过 Sentinel 感知新主并重连。只在同一机器放多个进程，机器故障时仍一起消失，数量不能代替独立的网络、电源与主机容错。
3. 网络分区可能使旧主一侧继续接收写入，恢复后这些写入可能丢失。min-replicas-to-write 等配置能限制一部分风险，但会降低副本不足时的可用性，不能作为所有分区情形的完整保证。

常见追问

追问：Sentinel 会转发业务请求吗？通常不会，它负责监控与发现，客户端仍直接连接当前 Redis 主节点。客户端必须支持发现、重连、超时与未知执行结果处理，否则服务器切换完成也可能无法恢复应用访问。

易错点

Sentinel 不是数据分片方案，也不是持久化替代物。故障恢复服务可用与保证每一笔已确认写入永久存在，是不同目标。

延伸阅读 / 原始参考资料

Redis 官方 • Sentinel

13. Cluster 的 16384 个槽与 hash tag 怎么用？

复制与集群 / 进阶

Redis Cluster 把键空间划为 16384 个 hash slot，再把槽分配给主节点。CRC16 与 hash tag 决定键所在槽，客户端负责识别重定向；多键原子操作通常要求同槽，分片也不会自动拆开一个热点键。

回答要点

1. 通常按 $\text{CRC16}(\text{key})$ 对 16384 取模计算槽。符合规则的非空花括号内容可作为 hash tag，例如 `order:{42}` 与 `lock:{42}` 会落到相同槽，有助协调相关键，但大范围共用标签会集中负载。
2. MOVED 表示客户端应更新该槽的目标节点认知；迁移期间 ASK 要按临时重定向流程处理，不能简单永久改写全部映射。使用支持 Cluster 的客户端，可处理拓扑变化并减少不必要重试。

3. 每个主节点负责一部分槽并可带副本，高可用仍受多数、槽覆盖和配置影响。Cluster 复制默认异步，故障切换有写入丢失窗口；跨槽操作不具备自动的全局事务，需要业务重新设计。

常见追问

追问：增加节点为什么不能解决单 key 热点？一个键仍由它所在槽的主节点处理，迁移槽只是转移瓶颈。可考虑本地缓存、读取副本或可合并的业务拆分，但拆键会改变原子聚合和一致性，需要按操作特征设计。

易错点

16384 是槽数，不是必须有 16384 台机器，也不是一致性哈希环的虚拟节点数。hash tag 能保证同槽，不能保证键分散或跨节点原子提交。

延伸阅读 / 原始参考资料

Redis 官方 • Cluster 规范

14. 缓存穿透、击穿与雪崩如何区分？

缓存与业务 / 基础

穿透是不存在的数据反复回源，击穿是热点失效引发并发回源，雪崩是大量失效或缓存服务故障导致整体压力突增。这些是工程场景术语，解决方案要匹配流量形态，并保护后端容量。

回答要点

1. 穿透可通过输入校验、短 TTL 的空值缓存及布隆过滤器减少无效回源。布隆过滤器可能误判存在，必须允许继续查权威数据；更新遗漏会影响新数据可见性，负缓存也应在创建后失效。
2. 击穿可用请求合并、短租约互斥或提前刷新降低同一热点的并发加载。若允许短暂旧数据，可使用逻辑过期后台更新，但必须设定允许陈旧时间、失败回退和重建者崩溃后的处理。
3. 雪崩可通过 TTL 抖动、热点预热、分批刷新及缓存高可用降低发生概率。缓存故障时不能把所有请求无限制压给数据库，应设置限流、超时、隔离与有意义的降级，并实测后端承载。

常见追问

追问：缓存空值有什么副作用？刚创建的数据可能被旧空结果遮蔽，恶意随机键还能制造大量负缓存占内存。应控制 TTL、键数量与输入范围，创建成功后主动失效相关负缓存，并根据访问模式决定是否需要布隆过滤。

易错点

术语不是严格协议定义，面试先说清触发条件。不要认为加一个分布式锁就解决所有缓存故障，它只协调特定键的重建，不能替代整体容量控制。

延伸阅读 / 原始参考资料

Redis 官方 • 旁路缓存

Redis 官方 • 布隆过滤器

15. MySQL 与 Redis 缓存如何保持一致？

缓存与业务 / 进阶

旁路缓存通常先更新数据库，再删除缓存，让后续读取重新加载。数据库与 Redis 没有天然原子提交，这个顺序仍可能存在竞态或失效失败；可用幂等重试、事务 outbox 或 CDC 收敛，并明确允许陈旧窗口。

回答要点

1. 读请求先查缓存，未命中再读数据库并回填；写请求提交数据库后使缓存失效。删除缓存失败需要可重试的失效任务，若直接同时覆盖数据库与缓存，乱序写入更容易让旧值覆盖新值。
2. 并发读可能在写提交前拿到旧值，却在写请求删缓存之后回填旧结果。TTL 限制一般旧值寿命，但长请求、不断刷新或异步失效延迟也会影响窗口，因此不能仅凭顺序宣称绝对一致。
3. 可把失效事件与业务修改写入同一数据库事务，再由可靠消费者重试删除；或通过 CDC 传播变化。消费者需幂等并处理乱序，关键读取可直接读权威数据库，版本化缓存也可防止旧值覆盖新值。

常见追问

追问：延迟双删一定安全吗？不是。第二次删除只能覆盖某个估计的并发窗口，读耗时、任务失败和调度延迟变化会突破固定等待。它可降低竞态概率，但严格要求需依赖可靠事件、版本控制或统一的权威读写边界。

易错点

不要承诺“先写数据库再删缓存就强一致”。该模式是实用的收敛方案，业务需要说明能容忍多久旧值，并给失效失败、读写竞态和恢复流程明确处理。

延伸阅读 / 原始参考资料

Redis 官方 • 旁路缓存实现与失效边界

16. Redis 分布式锁怎么写，有哪些不能保证的事？

缓存与业务 / 进阶

常见单实例锁用 `SET key token NX PX lease` 原子设置占有与租约，释放时原子比较令牌再删除。租约过期、进程停顿与异步故障切换可能让多个执行者重叠，锁本身不能保证外部资源永远只有一个写入者。

回答要点

1. 令牌需为每次获取唯一的不可混淆值，释放时不能直接 `DEL`，否则旧持有者恢复后可能删除新持有者的锁。比较与删除必须在一个短脚本或等价原子操作中完成，获取和过期也必须合并。
2. 租约比业务耗时短会过期，续租仍可能受停顿、网络分区或调度影响。客户端应检查是否继续拥有锁并限制任务时间，但已经开始的外部写入不会被 Redis 自动撤销，也不能靠锁状态终止一切旧任务。

3. 对关键外部资源，可使用单调 fencing token 让资源拒绝旧持有者写入，或由数据库约束和幂等实现最终正确性。随机锁令牌用于确认所有权，不能自动代替单调 fencing；多节点方案也有时钟和故障假设。

```
SET lock:job:42 unique-request-token NX PX 10000
-- 释放使用比较令牌后 DEL 的原子脚本
```

常见追问

追问：主从切换为何可能破坏互斥？主节点确认锁后，副本尚未收到该写入就发生故障，提升后的节点会认为锁不存在，于是另一个客户端可获取。WAIT 可降低某些风险但不能无条件消除；关键业务需更强的资源端约束或协调协议。

易错点

不要把 Redis 锁当资金、库存或任务唯一执行的唯一证明。其保证依赖部署、租约与故障模型，必须说明目标是减少重复工作还是保证外部状态严格互斥。

延伸阅读 / 原始参考资料

Redis 官方 • 分布式锁

17. Pipeline 为什么提速，和事务有何区别？

消息与运维 / 基础

Pipeline 连续发送一批命令再集中接收响应，主要减少网络往返和系统调用开销。它不提供事务式连续执行或回滚，也不能让有数据依赖的客户端计算自动在批内完成，应控制批量大小。

回答要点

1. 逐条请求需要每条等待往返，高延迟网络下即使命令很快总吞吐也受 RTT 限制。批量发送可摊薄网络等待，但服务端实际命令计算仍要执行，复杂度不会因 Pipeline 而从线性变为常数。
2. 一批请求会占用客户端和服务端的回复缓冲，过大批次可能增加内存、排队与尾延迟。按字节数和命令数控制批次，并考虑 Cluster 中路由到多个节点后需分别发送和聚合结果。
3. 连接断开时一部分命令可能已执行但响应丢失，批量重试必须考虑幂等。Pipeline 只是传输优化，若需检查后修改用 Lua 或合适事务，若需要拿到第一条结果才能决定第二条则要重新设计。

常见追问

追问：Pipeline 与 MGET 怎么选？多个独立 GET 可用 Pipeline 降低往返，同槽且适合的多个键可用 MGET 一条命令返回。Cluster 跨槽受命令与客户端能力限制，且大量返回数据仍占网络和内存，应根据拓扑与大小决定。

易错点

不要把 Pipeline 当作自动 MULTI/EXEC。不同客户端有的批量接口默认包装事务，必须确认具体库配置，不能从一个方法名推断实际原子语义。

延伸阅读 / 原始参考资料

Redis 官方 • Pipeline

18. Stream 消费者组如何实现确认与重试？

消息与运维 / 进阶

Stream 是可保留消息的日志结构，消费者组记录未确认消息。XREADGROUP 读取、XACK 确认，故障后可检查 pending 并认领重试；通常需按至少一次处理设计幂等，不能声称天然端到端恰好一次。

回答要点

1. XADD 写入带 ID 的条目，组内消费者通过 XREADGROUP 分配新消息。交付后条目进入 pending 状态，XACK 只是确认该组已处理，不等于立即从整个 Stream 删除，也不替业务事务提交。
2. 消费者崩溃后，消息不会自动在瞬间交给所有其他消费者。需要检查 XPENDING，并根据合理的空闲阈值用 XAUTOCLAIM 等认领；阈值过短会让慢但存活的消费者与认领者重复处理。
3. 先完成业务提交再确认可减少丢处理风险，但业务提交成功、确认前崩溃会导致再次交付，所以要用唯一消息标识去重。修剪策略、持久化与复制也决定消息保留，不合理 trim 可能删除仍需处理的内容。

```
XGROUP CREATE events workers 0 MKSTREAM
XREADGROUP GROUP workers consumer-1 COUNT 10 STREAMS events >
-- 业务成功并保证幂等后
XACK events workers <message-id>
```

常见追问

追问：Pub/Sub 与 Stream 的区别？Pub/Sub 给在线订阅者推送，通常没有历史保留和消费确认，离线期间消息会丢。Stream 可存储条目与组消费状态，适合重放和重试，但可靠性仍取决于持久化、保留配置与业务幂等。

易错点

XACK 不等于删除消息，也不等于业务只执行一次。消费确认与外部数据库提交是两个系统动作，必须处理两者之间的崩溃窗口。

延伸阅读 / 原始参考资料

Redis 官方 • 消费者组读取

Redis 官方 • 认领 pending 消息

Redis 官方 • 确认消息

19. 大 key 与热 key 如何发现和治理？

消息与运维 / 进阶

大 key 关注单键字节数或集合元素多，热 key 关注访问频率高；两者可能重叠但处理不同。大 key 放大遍历、网络与删除成本，热 key 集中节点负载，应结合采样、命令统计和业务指标定位。

回答要点

1. 用 MEMORY USAGE 等了解单键内存，结合 SCAN 或工具抽样发现异常集合；全量 KEYS 会扫描并阻塞，不适合高负载线上排查。SCAN 分批更温和，但不提供静态一致快照且可能返回重复。
2. 大集合操作尽量限量与分批，避免 HGETALL、全集合运算或一次返回巨量成员。删除可用 UNLINK 将内存回收工作移到后台，但对象分离和异步释放仍有成本，不意味着内存立刻下降。
3. 热读可通过本地缓存、请求合并或容忍旧值的副本读缓解；热写可能需业务可合并的分片计数。单键在 Cluster 仍归一个槽主节点，增加节点只能扩大总容量，不会天然均匀摊开该键操作。

常见追问

追问：什么大小才算大 key？没有统一阈值，要结合命令复杂度、网络带宽、最大延迟和节点负载。一个几 MB 的 String 与几十万成员的 Hash 风险不同，应制定按类型、元素数、总字节和单次返回量区分的预算。

易错点

不要为了拆 key 破坏原子操作或引入更多小对象内存开销。治理前先确认瓶颈是字节量、成员遍历还是访问频率，并明确拆分后的聚合和一致性方式。

延伸阅读 / 原始参考资料

Redis 官方 • 内存用量

Redis 官方 • 渐进扫描

Redis 官方 • 异步释放

20. 如何评估 Redis 的可靠性与尾延迟？

消息与运维 / 进阶

可靠性由持久化、复制、切换与业务重试共同决定，尾延迟则受慢命令、fork、内存和网络影响。应明确允许的数据丢失与恢复时间，监控实际指标，并通过故障演练验证，而不是只看平均延迟与服务存活。

回答要点

1. 区分客户端确认、写入内存、日志追加、fsync 和副本确认几个阶段。WAIT 等待复制确认，WAITAOF 在支持版本中可等待相关 AOF 同步，但都应检查返回结果与超时，不能替代完整恢复验证。
2. SLOWLOG 关注命令执行耗时，通常不包含与客户端通信的全部时间；应用观察的延迟还包括排队、网络和解析。结合延迟监控、命令统计、fork 耗时、内存碎片、持久化状态与复制延迟找具体原因。
3. 故障演练覆盖进程退出、主机故障、网络分区、磁盘满和重启装载，记录恢复耗时与丢写结果。备份要跨故障域保存并实际恢复，业务超时重试要幂等，否则恢复后仍可能出现重复扣减或任务执行。

常见追问

追问：开了 AOF 加一个副本就一定不丢吗？不一定。刷盘策略、复制异步、共同故障域、自动切换选择与设备行为都影响结果。先定义 RPO/RTO，再选择配置、确认方式、备份和业务容错，并用实测说明保证范围。

易错点

不要把服务自动恢复称为数据绝不丢失，也不要将平均 1ms 视为每次 1ms。高峰、持久化任务和异常负载下的 P99 往往决定用户体验。

延伸阅读 / 原始参考资料

Redis 官方 • 慢命令日志

Redis 官方 • AOF 同步等待

Redis 官方 • 运行指标